DevOps – Lab 05

Build Tools & Artifact Repositories: Maven, Gradle, Node.js/npm and a Jenkins Pipeline

Solution Report with Screenshots

Task	Topic	Figures
1	IntelliJ IDEA environment (Docker + SSH tunnel)	1–2
2	Maven project – java-maven-app	3–7
3	Gradle project – java-app	8–14
4	Node.js project – node-project	15–21
5	React / Node.js API artifact – react-nodejs-example	22–25
6	Jenkins pipeline (Jenkinsfile)	26–28

Task 1 – IntelliJ IDEA Environment

Objective: Prepare a Java IDE to clone, build and run the lab projects. IntelliJ IDEA runs inside a Docker container (`intellij-idea-docker-intellij-1`) on an Ubuntu 24.04 VM and is reached from the Windows host in a browser through an SSH tunnel (ports 3001 and 3000).

Steps: (1) Open the tunnel from PowerShell: `ssh -L 3001:localhost:3001 -L 3000:localhost:3000 user@192.168.154.128`. (2) Browse to `https://localhost:3001` to reach the IntelliJ welcome screen. (3) Clone the Docker/IntelliJ repository on the VM.

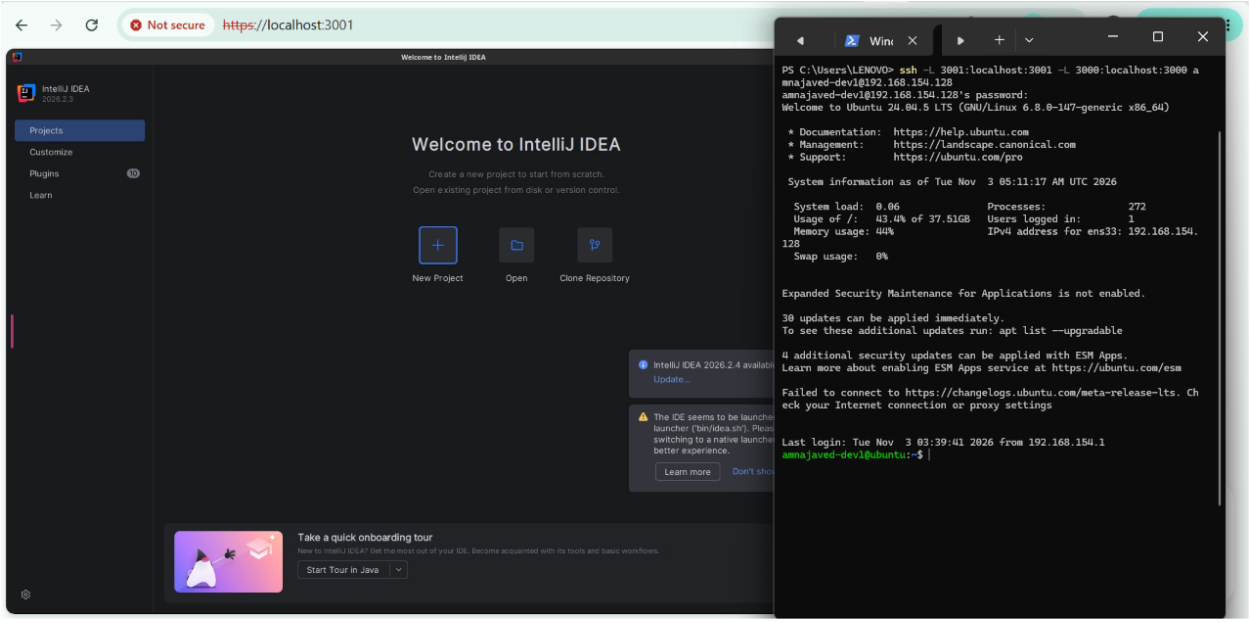


Figure 1 – IntelliJ IDEA welcome screen in the browser (left) and the SSH tunnel session to the Ubuntu VM (right). (intellij_desktop.png)

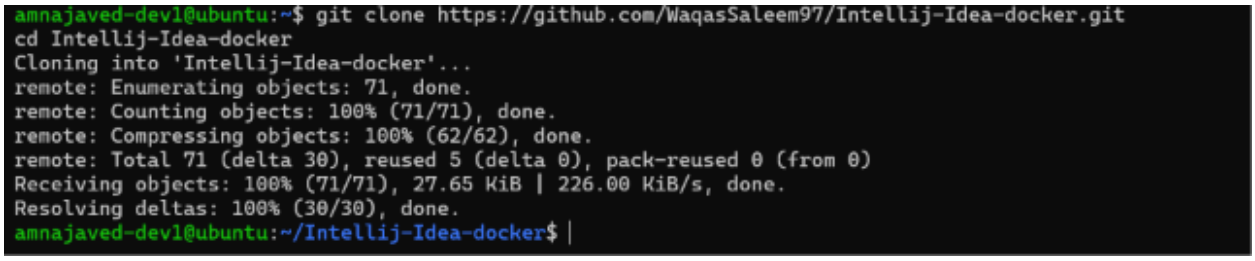


Figure 2 – Cloning the Intellij-Idea-docker repository with git on the VM. (intellij_repo_clone.png)

Task 2 – Maven Project (java-maven-app)

Objective: Clone a Java Maven project, install Maven, add the Elasticsearch dependency, then compile/package and test it.

2.1 Clone the project

Using IntelliJ *Clone Repository* with URL `https://gitlab.com/nanuchi/java-maven-app.git`; the project opens with `src`, `pom.xml`, `Jenkinsfile` and `script.groovy`.

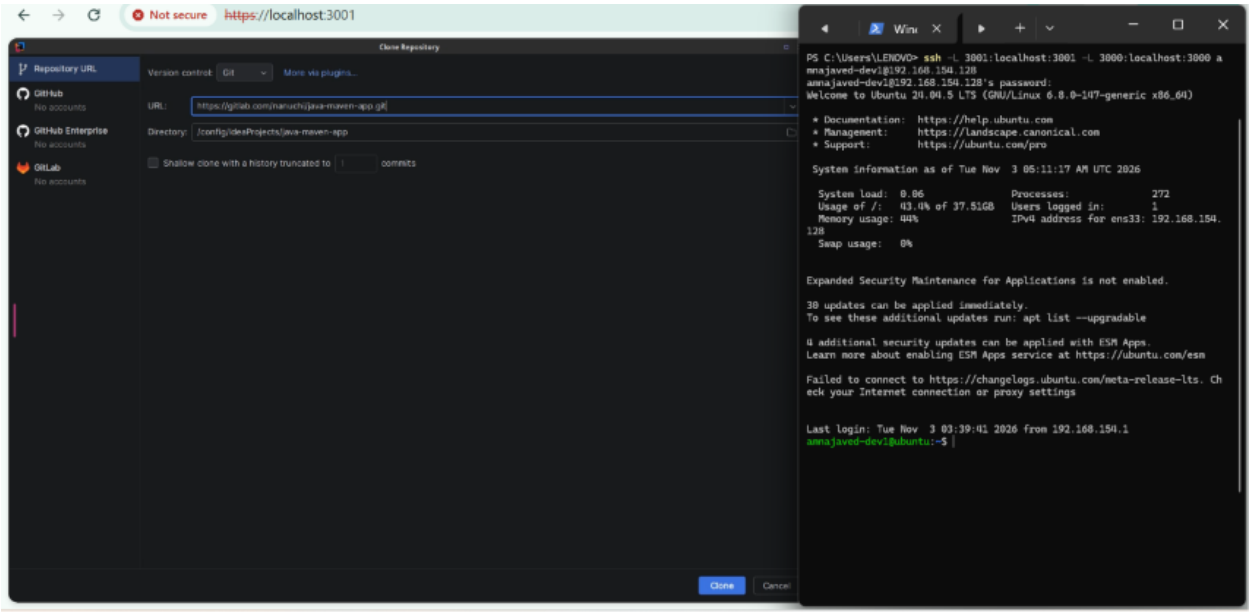


Figure 3 – Clone Repository dialog for java-maven-app (directory /config/IdeaProjects/java-maven-app). (maven_project_clone.png)

2.2 Install Maven

Maven was installed with apt and verified in the IntelliJ terminal: `mvn --version` reports Apache Maven 3.9.12 on OpenJDK 21.

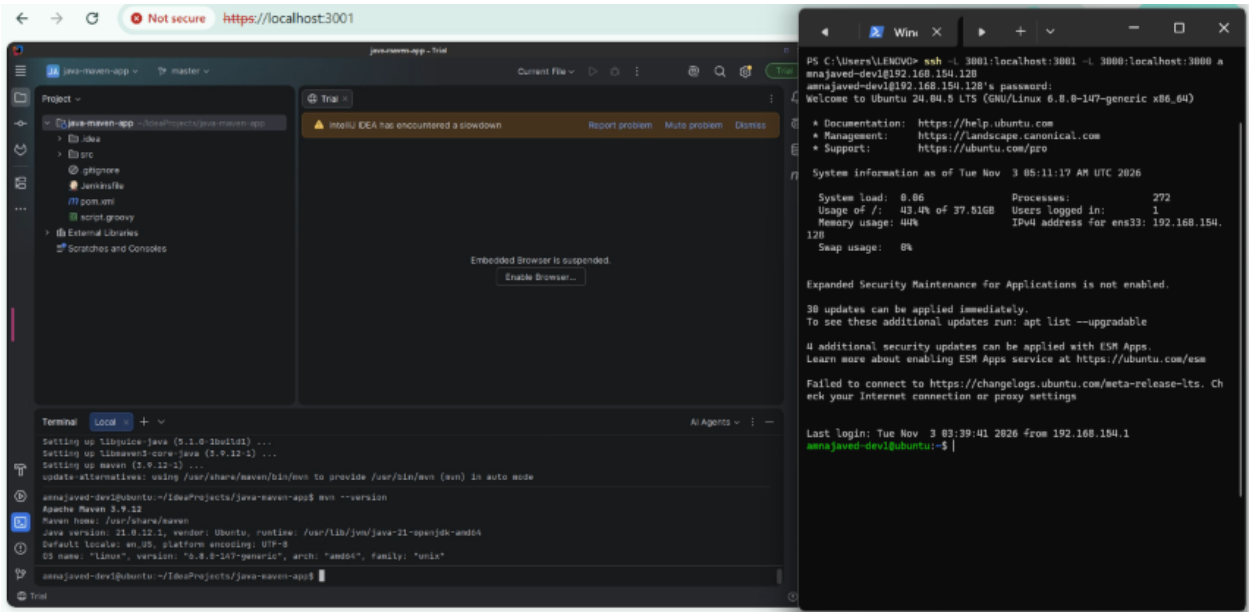


Figure 4 – Project loaded in IntelliJ and *mvn --version* output in the terminal. (maven_installed.png)

2.3 Add the Elasticsearch dependency

Added to `pom.xml`:

```
<dependency>
    <groupId>org.elasticsearch</groupId>
    <artifactId>elasticsearch</artifactId>
    <version>7.0.0</version>
    <scope>compile</scope>
</dependency>
```

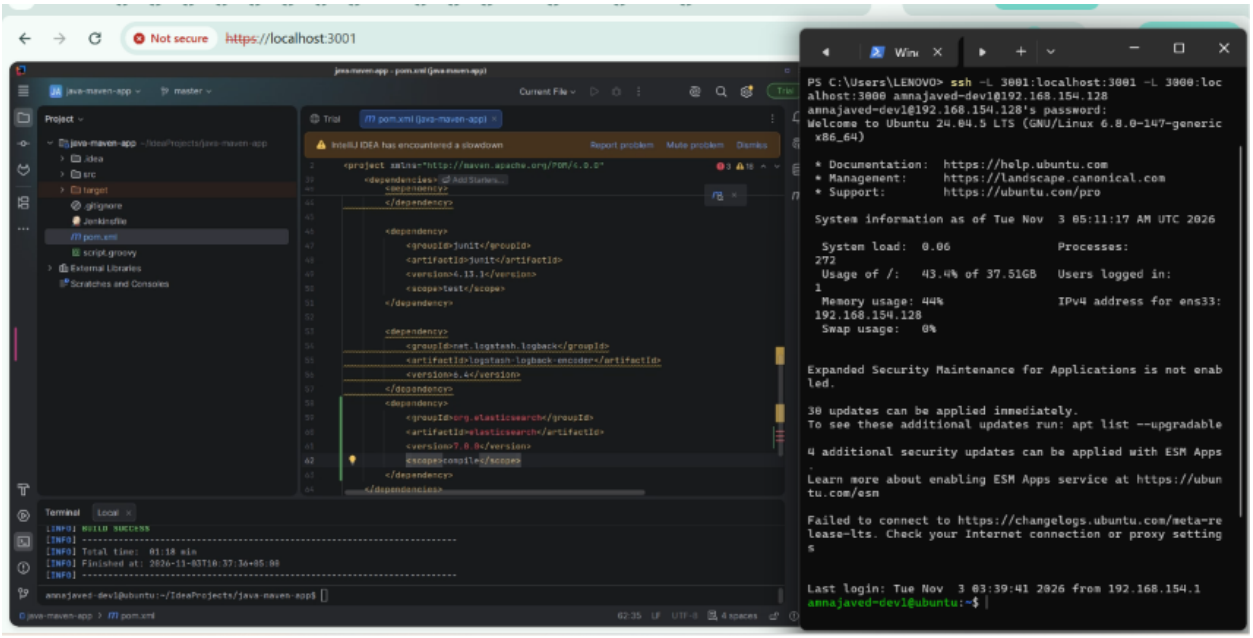


Figure 5 – pom.xml with the org.elasticsearch:elasticsearch:7.0.0 dependency added; the earlier build finished with BUILD SUCCESS. (maven_elasticsearch_dependency.png)

2.4 Package the application

`mvn package` downloaded the dependencies and replaced the main artifact with the repackaged archive in `target/`. Result: **BUILD SUCCESS** (about 1 min 18 s).

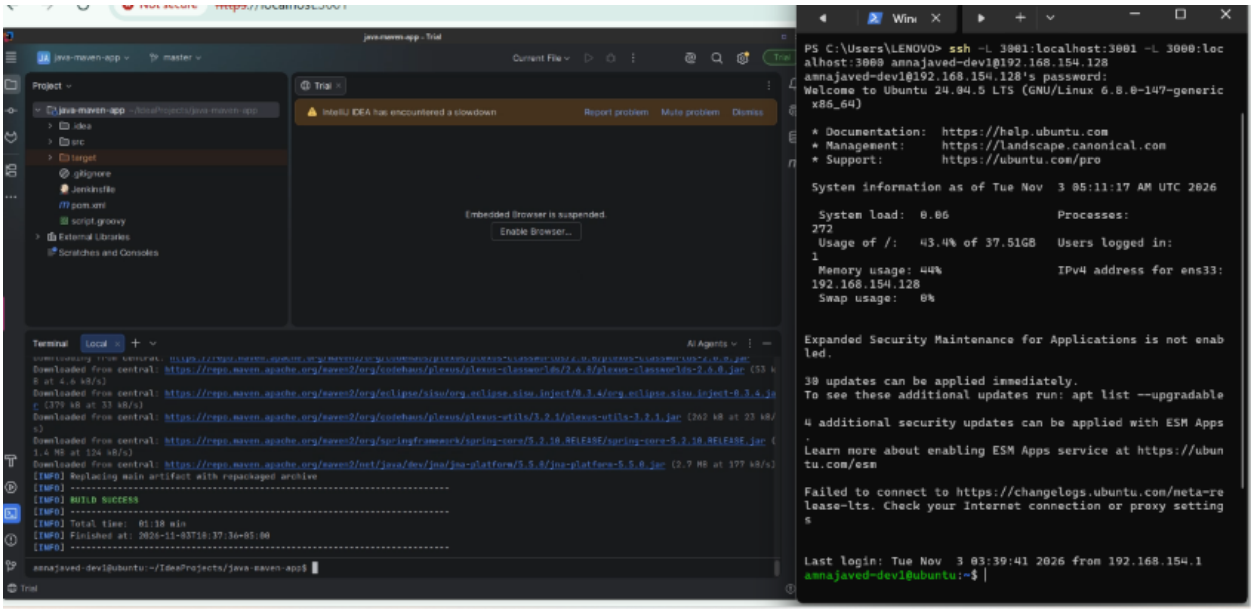


Figure 6 – mvn package – dependencies downloaded, BUILD SUCCESS, target/ directory created. (maven_package.png)

2.5 Run the tests

`mvn test` ran the full lifecycle up to the surefire plugin (resources, compile, test-compile, test). The project has no unit tests so it reports “No tests to run”, and the build ends with **BUILD SUCCESS** (12.2 s).

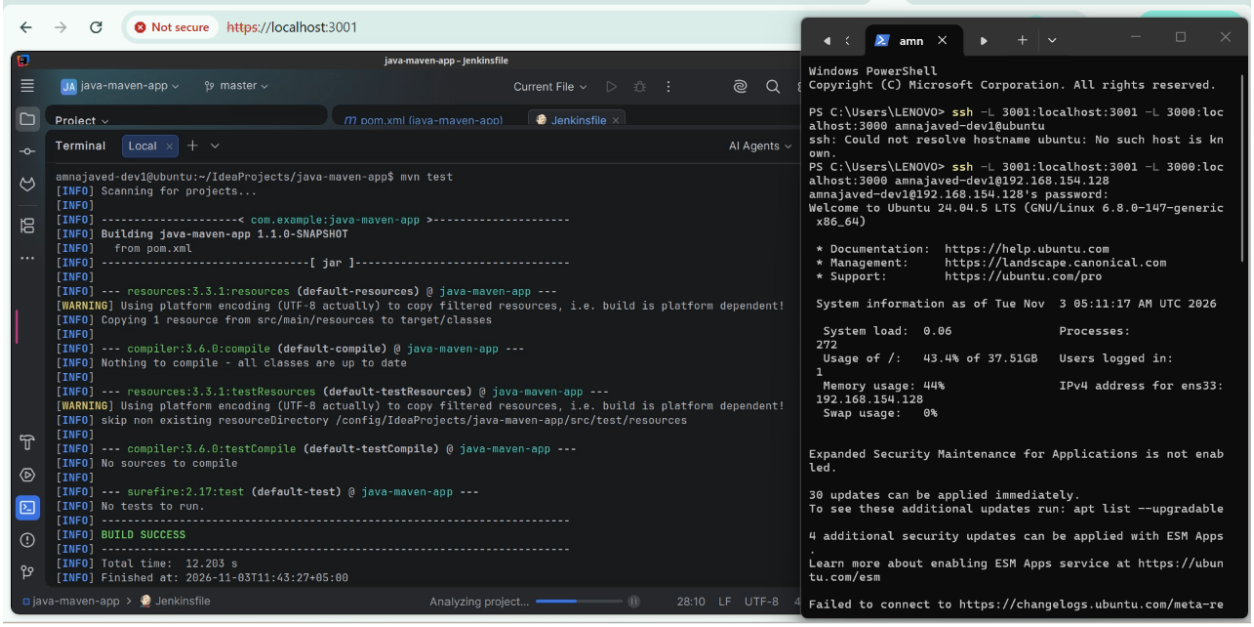


Figure 7 – mvn test output – BUILD SUCCESS. (maven_test.png)

Task 3 – Gradle Project (java-app)

Objective: Clone a Gradle project, add the Elasticsearch dependency, build it, produce the JAR, inspect Gradle tasks and rebuild. Commands were run inside the IntelliJ container with `docker exec`.

3.1 Clone the project

Cloned from `https://gitlab.com/nanuchi/java-app.git` to `/config/IdeaProjects/java-app`.

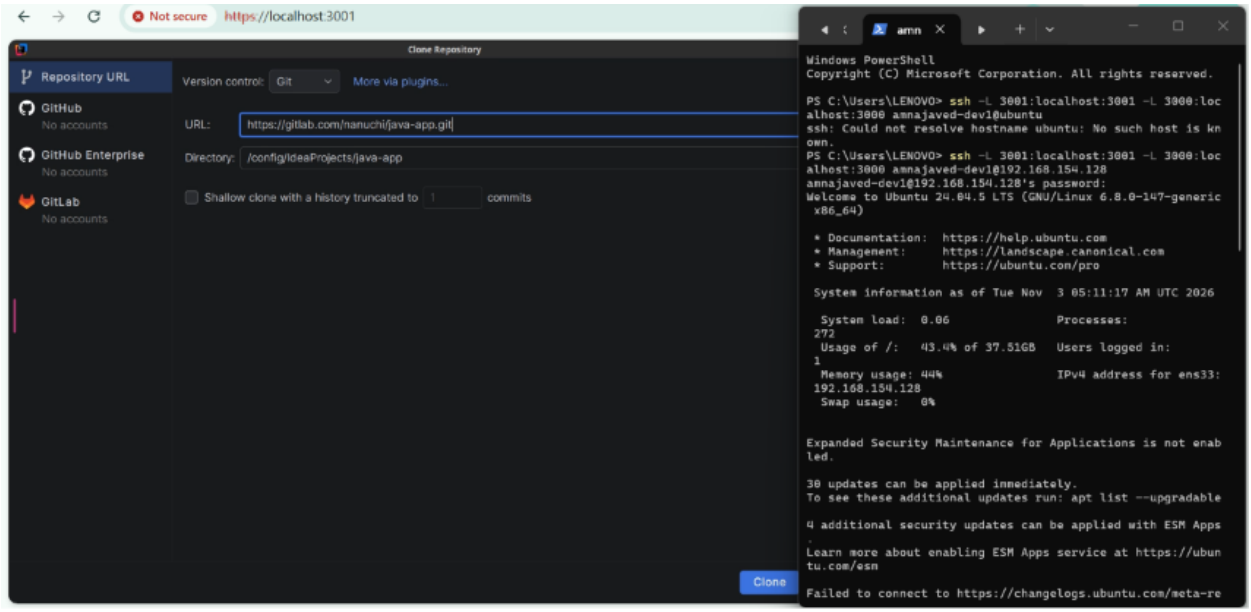


Figure 8 – Clone Repository dialog for java-app. (gradle_project_clone.png)

3.2 Java version and Elasticsearch dependency

Java 21 (OpenJDK 21.0.12.1) is available in the container. `grep` on `build.gradle` confirms line 15 declares the dependency:

```
implementation("org.elasticsearch:elasticsearch:7.0.0")
```

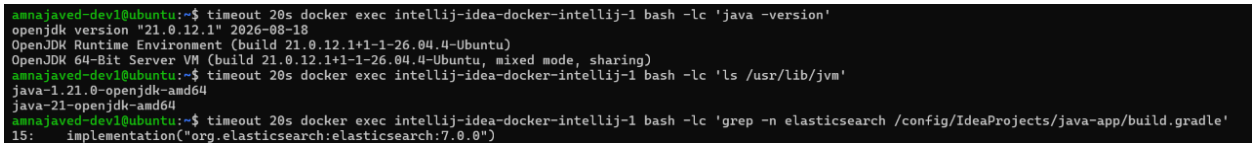


Figure 9 – java -version, installed JVMs and the Elasticsearch dependency in build.gradle. (gradle_elasticsearch_dependency.png)

3.3 Build the project

`./gradlew build` downloaded Gradle 8.5 through the wrapper, compiled the Java sources and finished with **BUILD SUCCESSFUL in 14m 10s** (3 actionable tasks executed).

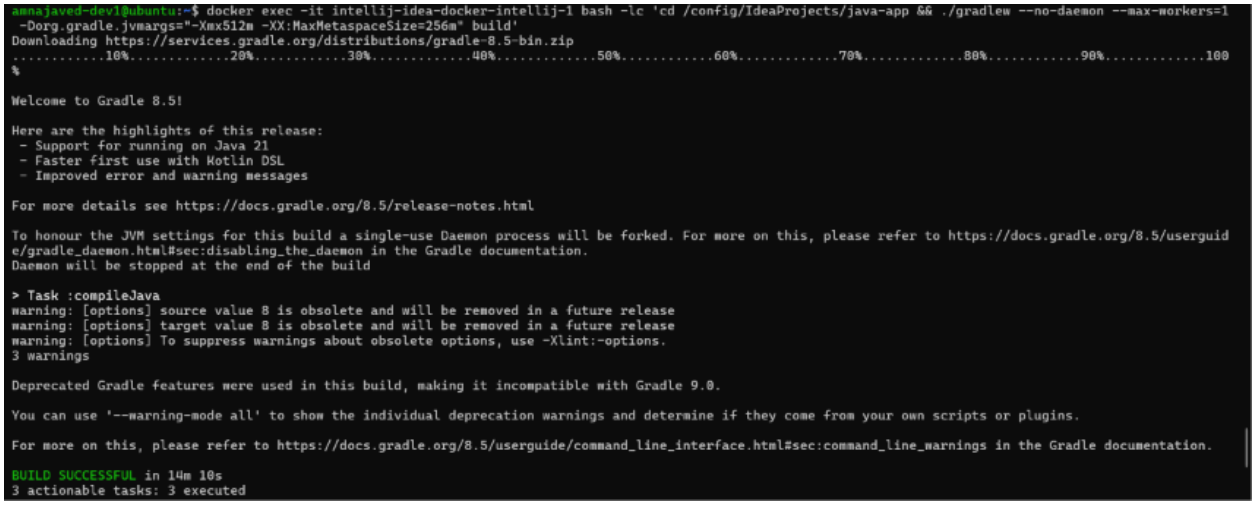


Figure 10 – ./gradlew build – BUILD SUCCESSFUL. (gradle_build.png)

3.4 Generated JAR

The build produced `java-app-1.0-SNAPSHOT.jar` (1744 bytes) in `build/libs`.

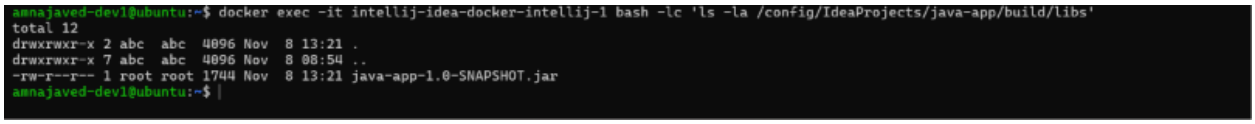


Figure 11 – Listing of build/libs showing java-app-1.0-SNAPSHOT.jar. (gradle_jar.png)

3.5 Gradle tasks

`./gradlew tasks` lists the runnable tasks: build tasks (assemble, build, clean, jar, classes), build-setup tasks (init, wrapper), documentation (javadoc) and help tasks.

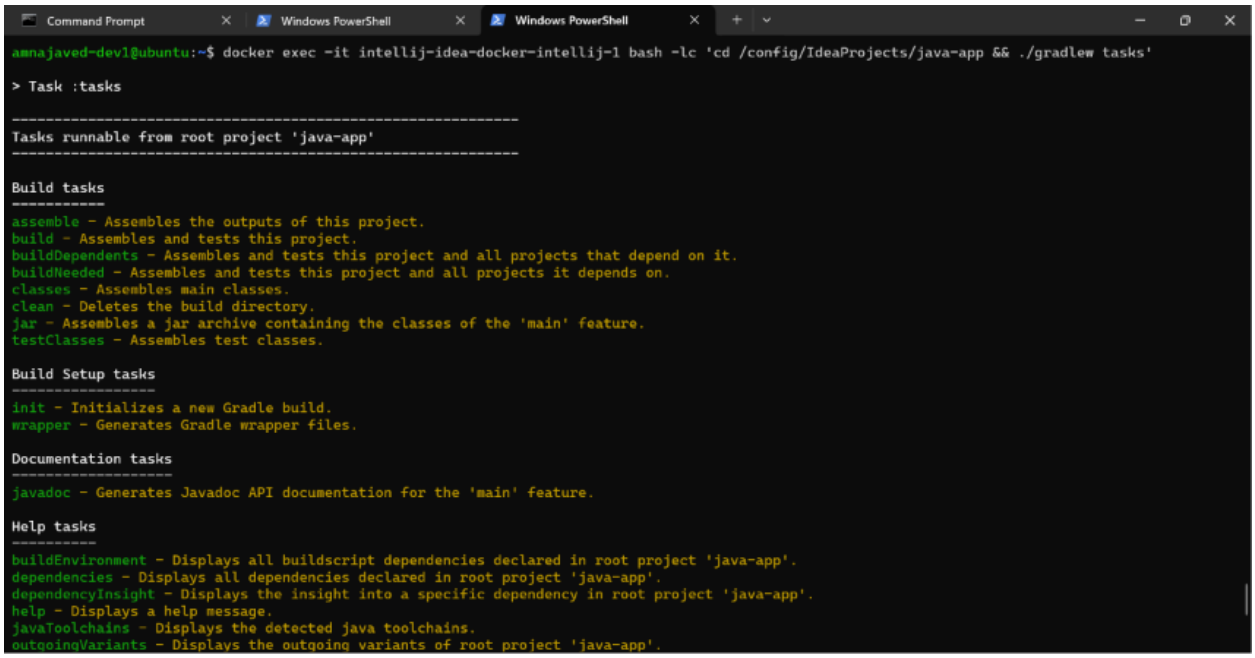


Figure 12 – ./gradlew tasks – available Gradle tasks for the java-app project. (gradle_running.png)

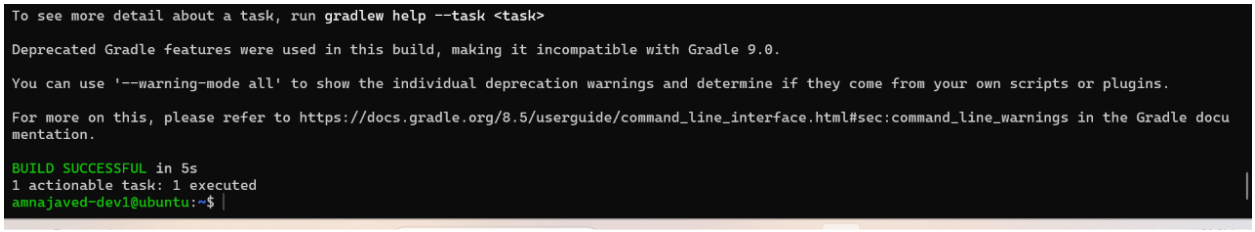


Figure 13 – End of the tasks run – BUILD SUCCESSFUL in 5s; the single-use daemon stops when the build ends. (gradle_stopped.png)

3.6 Rebuild

A second `./gradlew build` completed in 29 s with all 3 tasks **up-to-date**, showing Gradle's incremental build: nothing was recompiled because nothing changed.

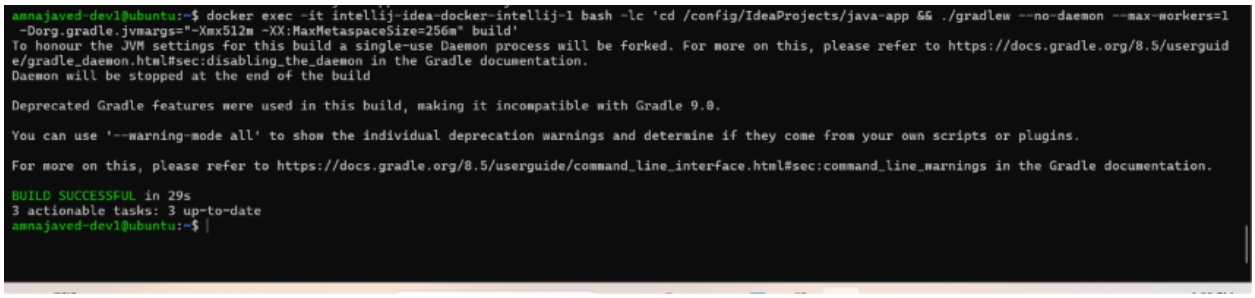


Figure 14 – Rebuild – BUILD SUCCESSFUL in 29s, 3 actionable tasks up-to-date. (gradle_rebuild.png)

Task 4 – Node.js Project (node-project)

Objective: Clone a Node.js project, install Node.js and npm, install the dependencies, run and test the app, and package it as an artifact with `npm pack`.

4.1 Clone the project

Cloned from `https://gitlab.com/devopsbootcamp3/node-project.git`. The repository contains `app/`, `Dockerfile`, `Jenkinsfile`, `README.md` and `.gitignore`.

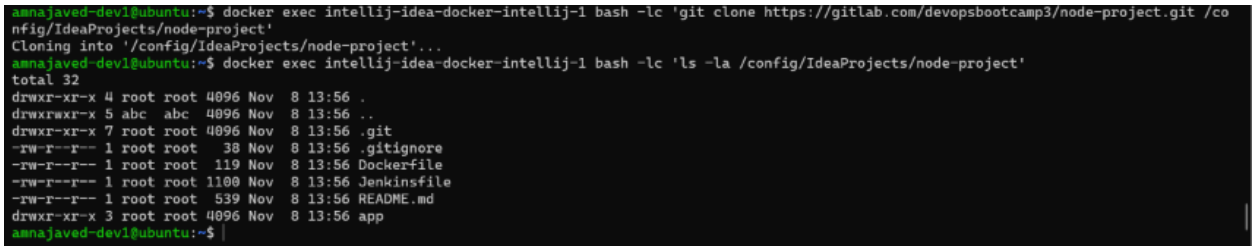


Figure 15 – git clone of node-project and directory listing. (node_project_clone.png)

4.2 Install Node.js and npm

Installed with `apt install -y nodejs` from the NodeSource repository (Node 22). Verified: **Node v22.23.4**, **npm 10.9.9**.

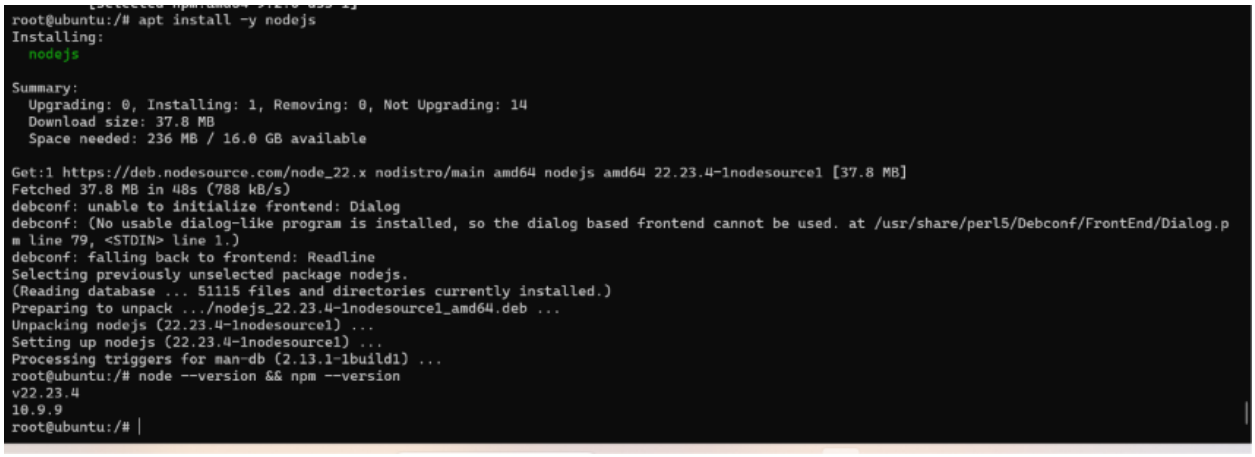


Figure 16 – Node.js installation and version check. (nodejs_installed.png)

4.3 Install dependencies

`npm install` inside `app/` added 554 packages (audited 555). npm printed deprecation warnings and an audit summary, which do not stop the install.

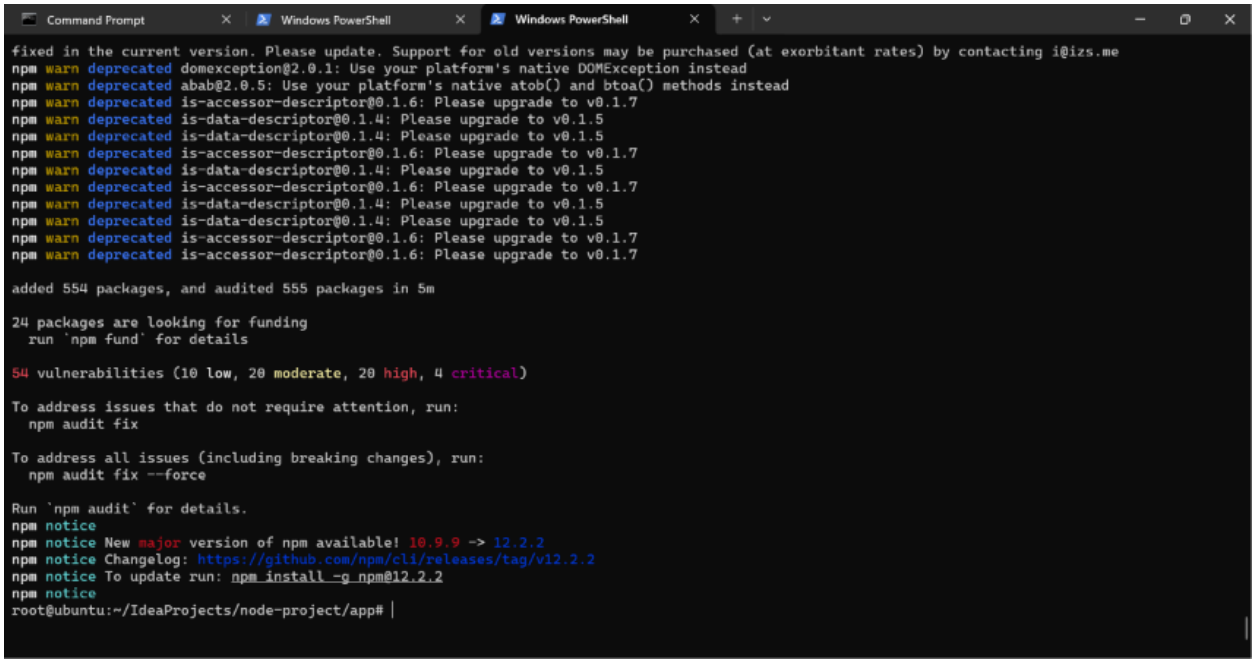


Figure 17 – npm install – 554 packages added. (npm_install.png)

4.4 Run the application

`npm start` runs `node server.js`; the app listens on **port 4000**.

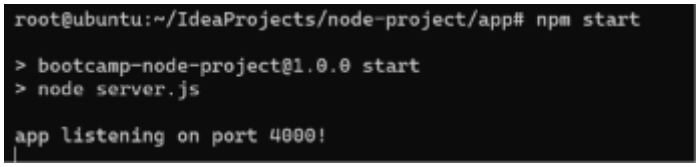


Figure 18 – npm start – “app listening on port 4000!”. (node_running.png)

4.5 Verify in the browser / HTTP

A request to `http://localhost:4000` returns **HTTP/1.1 200 OK** from Express (Content-Type text/html), confirming the app is served.

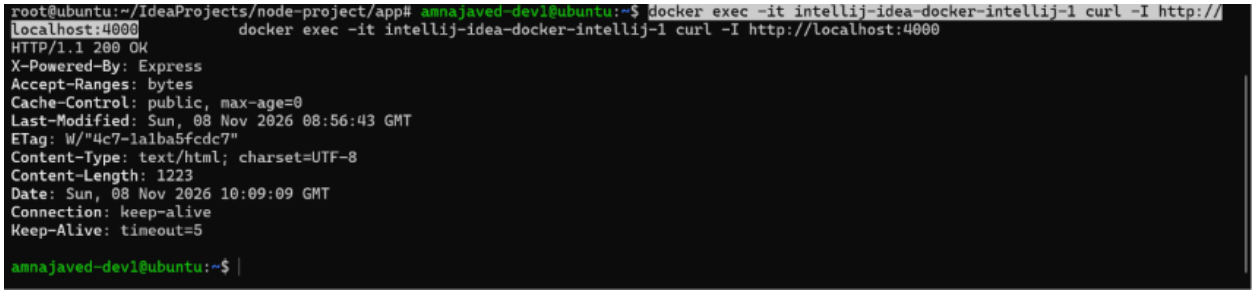


Figure 19 – curl -I http://localhost:4000 – HTTP 200 OK. (node_browser.png)

4.6 Run the tests

`npm test` runs Jest: **1 suite passed, 1 test passed** (“main index.html file exists”).

```
amnajaved-dev1@ubuntu:~$ cd /config/IdeaProjects/node-project/app
-bash: cd: /config/IdeaProjects/node-project/app: No such file or directory
amnajaved-dev1@ubuntu:~$ docker exec -it intellij-idea-docker-intellij-1 bash
root@ubuntu:/# cd /config/IdeaProjects/node-project/app
root@ubuntu:~/IdeaProjects/node-project/app# npm test

> bootcamp-node-project@1.0.0 test
> jest

Browserslist: caniuse-lite is outdated. Please run:
npx browserslist@latest --update-db

Why you should do it regularly:
https://github.com/browserslist/browserslist#browsers-data-updating
 PASS  ./server.test.js
  ✓ main index.html file exists (6 ms)

Test Suites: 1 passed, 1 total
Tests:       1 passed, 1 total
Snapshots:   0 total
Time:        5.499 s
Ran all test suites.
root@ubuntu:~/IdeaProjects/node-project/app#
```

Figure 20 – npm test – server.test.js PASS. (npm_test.png)

4.7 Package the artifact

`npm pack` bundles index.html, server.js, server.test.js, package.json and the images into `bootcamp-node-project-1.0.0.tgz` (79.4 kB, 6 files).

```
root@ubuntu:~/IdeaProjects/node-project/app# npm pack
npm notice
npm notice 📦  bootcamp-node-project@1.0.0
npm notice Tarball Contents
npm notice 60.5kB images/profile-andrea.jpg
npm notice 17.5kB images/profile-ari.jpeg
npm notice 1.2kB index.html
npm notice 283B package.json
npm notice 724B server.js
npm notice 205B server.test.js
npm notice Tarball Details
npm notice name: bootcamp-node-project
npm notice version: 1.0.0
npm notice filename: bootcamp-node-project-1.0.0.tgz
npm notice package size: 79.4 kB
npm notice unpacked size: 80.4 kB
npm notice shasum: 5fc88c5b26659efe277d2b6d5ded6ffd7f149029
npm notice integrity: sha512-m5e/IyK0Cmjw0[...]afe3T4Ix79dpA==
npm notice total files: 6
npm notice
bootcamp-node-project-1.0.0.tgz
root@ubuntu:~/IdeaProjects/node-project/app#
```

Figure 21 – npm pack – tarball contents and details. (npm_pack.png)

Task 5 – React / Node.js API Artifact

Objective: Build a deployable artifact of the Node.js API in the `react-nodejs-example` project using npm and webpack.

5.1 Clone the project

Cloned from `https://github.com/bbachi/react-nodejs-example.git` to `/config/IdeaProjects/react-nodejs-example` (5936 objects).

```
^Camnajaved-dev1@ubuntu:~$ docker exec -it intellij-idea-docker-intellij-1 bash -lc 'git clone https://github.com/bbachi/react-nodejs-example.gi
t /config/IdeaProjects/react-nodejs-example'
Cloning into '/config/IdeaProjects/react-nodejs-example'...
remote: Enumerating objects: 5936, done.
remote: Counting objects: 100% (645/645), done.
remote: Compressing objects: 100% (542/542), done.
remote: Total 5936 (delta 161), reused 103 (delta 103), pack-reused 5291 (from 2)
Receiving objects: 100% (5936/5936), 6.46 MiB | 256.00 KiB/s, done.
Resolving deltas: 100% (1246/1246), done.
amnajaved-dev1@ubuntu:~$
```

Figure 22 – git clone of react-nodejs-example. (react_node_project_clone.png)

5.2 Install the API dependencies

`npm install` in the `api` folder. The old lockfile and deprecated-package warnings are expected for this older project.

```
Command Prompt | Windows PowerShell | Windows PowerShell | amnajaved-dev1@ubun | amnajaved-dev1@ubun

amnajaved-dev1@ubuntu:~$ docker exec -it intellij-idea-docker-intellij-1 bash -lc 'cd /config/IdeaProjects/react-nodejs-example/api && npm insta
ll'
npm warn old lockfile
npm warn old lockfile The package-lock.json file was created with an old version of npm,
npm warn old lockfile so supplemental metadata must be fetched from the registry.
npm warn old lockfile
npm warn old lockfile This is a one-time fix-up, please be patient...
npm warn old lockfile
npm warn deprecated is-data-descriptor@0.1.4: Please upgrade to v0.1.5
npm warn deprecated inflight@1.0.6: This module is not supported, and leaks memory. Do not use it. Check out lru-cache if you want a good and te
sted way to coalesce async requests by a key value, which is much more comprehensive and powerful.
npm warn deprecated is-accessor-descriptor@0.1.6: Please upgrade to v0.1.7
npm warn deprecated is-data-descriptor@1.0.0: Please upgrade to v1.0.1
npm warn deprecated is-accessor-descriptor@1.0.0: Please upgrade to v1.0.1
npm warn deprecated is-data-descriptor@1.0.0: Please upgrade to v1.0.1
npm warn deprecated is-data-descriptor@1.0.0: Please upgrade to v1.0.1
npm warn deprecated is-data-descriptor@1.0.0: Please upgrade to v1.0.1
npm warn deprecated is-accessor-descriptor@1.0.0: Please upgrade to v1.0.1
npm warn deprecated is-accessor-descriptor@1.0.0: Please upgrade to v1.0.1
npm warn deprecated is-accessor-descriptor@1.0.0: Please upgrade to v1.0.1
npm warn deprecated urix@0.1.0: Please see https://github.com/lydell/urix#deprecated
npm warn deprecated rimraf@3.0.2: Rimraf versions prior to v4 are no longer supported
npm warn deprecated resolve-url@0.2.1: https://github.com/lydell/resolve-url#deprecated
npm warn deprecated source-map-resolve@0.5.3: See https://github.com/lydell/source-map-resolve#deprecated
npm warn deprecated source-map-url@0.4.0: See https://github.com/lydell/source-map-url#deprecated
npm warn deprecated rimraf@2.7.1: Rimraf versions prior to v4 are no longer supported
npm warn deprecated querystring@0.2.0: The querystring API is considered Legacy. new code should use the URLSearchParams API instead.
npm warn deprecated path-dirname@1.0.2: Package no longer supported. Contact Support at https://www.npmjs.com/support for more info.
npm warn deprecated lodash.clone@4.5.0: This package is deprecated. Use structuredClone instead.
npm warn deprecated move-concurrently@1.0.1: This package is no longer supported.
npm warn deprecated ini@1.3.5: Please update to ini >=1.3.6 to avoid a prototype pollution issue
npm warn deprecated fs-write-stream-atomic@1.0.10: This package is no longer supported.
npm warn deprecated glob@7.1.6: Old versions of glob are not supported, and contain widely publicized security vulnerabilities, which have been
fixed in the current version. Please update. Support for old versions may be purchased (at exorbitant rates) by contacting i@izs.me
npm warn deprecated figgy-pudding@3.5.2: This module is no longer supported.
npm warn deprecated default-resolution@2.0.0: Package no longer supported. Contact Support at https://www.npmjs.com/support for more info.
```

Figure 23 – npm install in react-nodejs-example/api. (react_api_dependencies.png)

5.3 Build the API bundle

`NODE_OPTIONS=--openssl-legacy-provider npm run build` runs webpack 4.43.0. The legacy OpenSSL option is needed because webpack 4 uses an MD4 hash that newer Node versions disable. Output: **server.bundle.js (563 KiB)**.

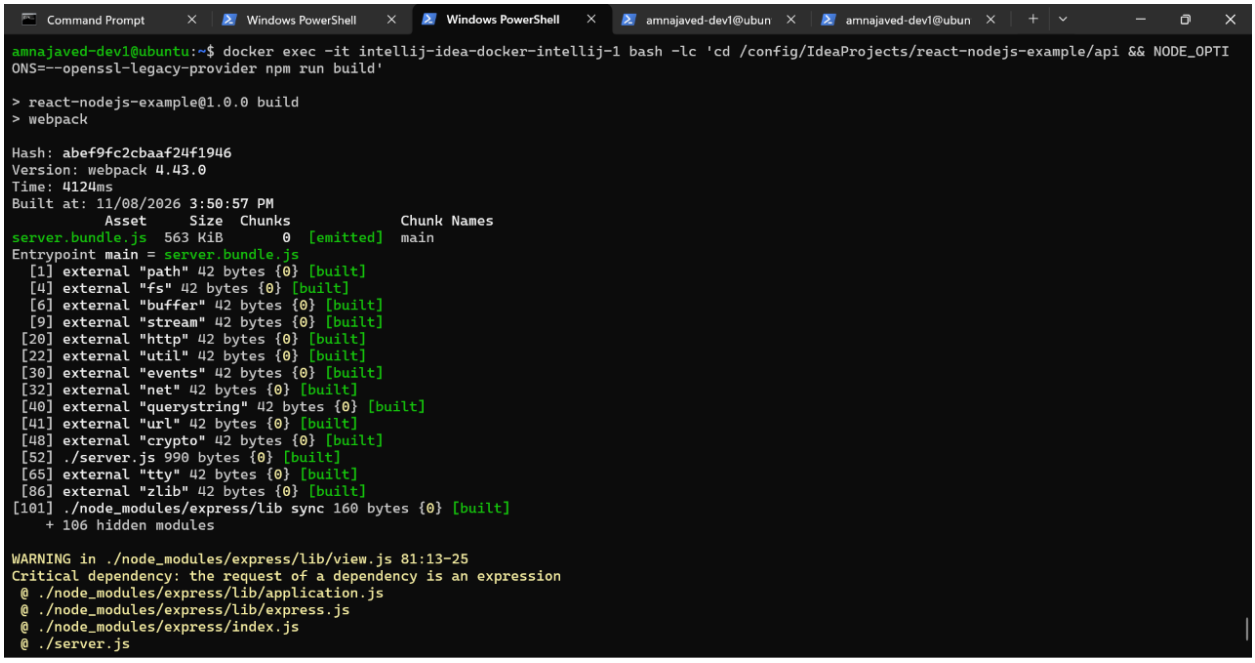


Figure 24 – Webpack build of the API – server.bundle.js emitted. (react_api_build.png!.png)

5.4 Inspect the bundle

`cat server.bundle.js` shows the minified bundle with the Express code and its dependencies packed into a single deployable file.



Figure 25 – Contents of the generated server.bundle.js. (react_api_bundle.png)

Task 6 – Jenkins Pipeline

Objective: Review the Jenkinsfile stages and run the pipeline's commands for the Node.js project.

6.1 Jenkinsfile (java-maven-app)

The Maven project's Jenkinsfile defines stages *build jar*, *build image* and *deploy*, each calling a function from `script.groovy` (`gv.buildJar()`, `gv.buildImage()`, `gv.deployApp()`).

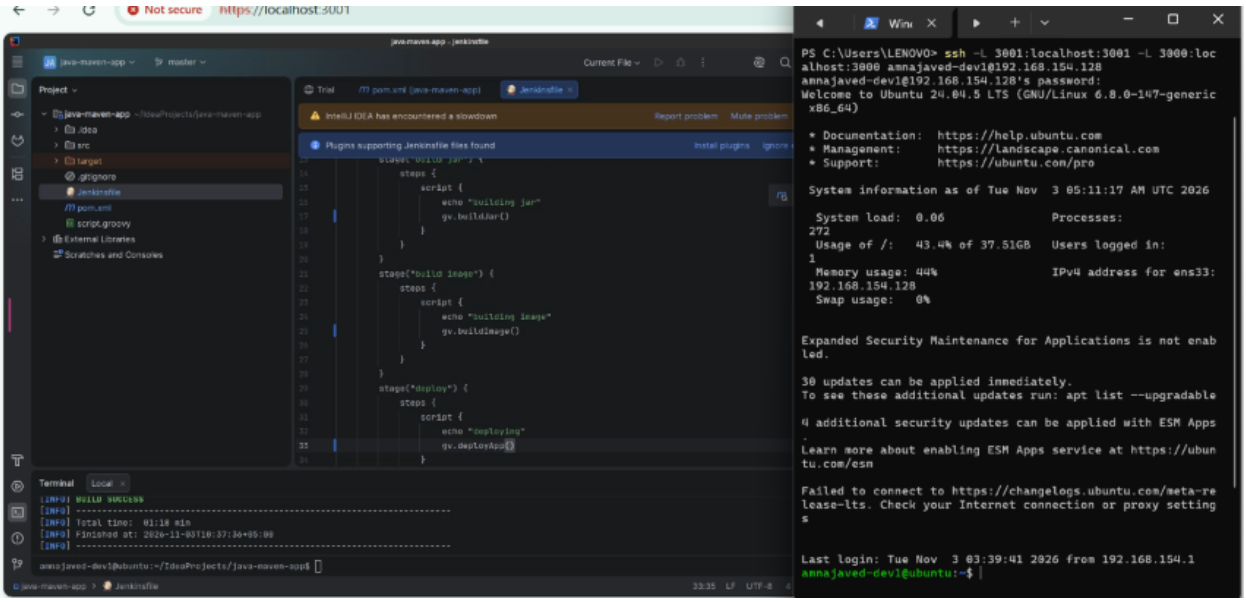


Figure 26 – Jenkinsfile of java-maven-app opened in IntelliJ. (jenkinsfile_overview.png)

6.2 Jenkinsfile (node-project)

The Node.js pipeline uses `agent any` and the `nodejs 'my-nodejs'` tool. Its *test* stage runs `cd app/; npm install` and `cd app/; npm test`; the *build jar* and *build image* stages are placeholders.

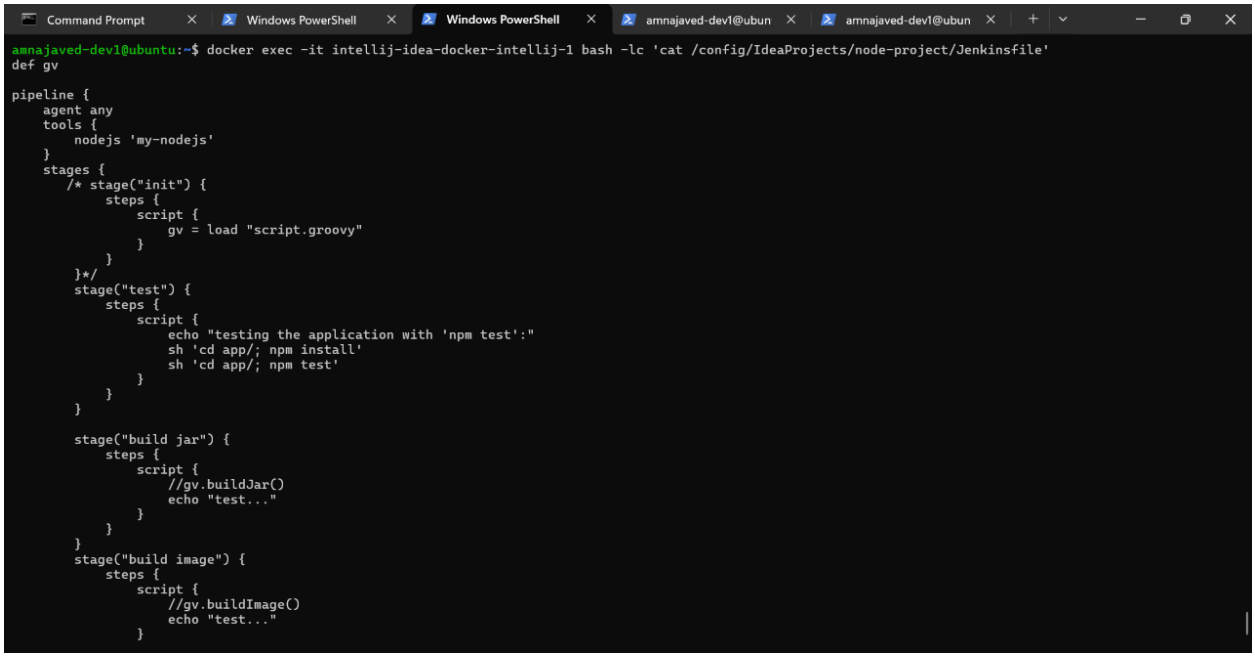


Figure 27 – Contents of the node-project Jenkinsfile. (jenkins_pipeline_file.png)

6.3 Pipeline test stage result

Running the test stage's commands (`npm install && npm test`) in the project: dependencies up to date, Jest **PASS**, 1 of 1 tests passed.



Figure 28 – Test-stage commands from the Jenkinsfile – tests pass. (jenkins_pipeline_success.png)

Conclusion

Maven, Gradle and npm were each used to resolve dependencies, build, test and package an application. Maven produced a repackaged JAR in `target/`, Gradle produced `java-app-1.0-SNAPSHOT.jar`, npm produced `bootcamp-node-project-1.0.0.tgz`, and webpack produced `server.bundle.js`. The Jenkinsfile stages automate the same steps in a CI pipeline.